

3 - Remediation documentation

Remediation Documentation — Project AEGIS

1. REST API — BOLA + Mass Assignment

Vulnerable Behavior

The `POST /api/profile/preferences` endpoint accepts an `employee_id` field directly from the request body and uses it to select which employee record to update, instead of deriving the target from the authenticated session. It also merges every field present in the request body onto the target record with no field whitelist, allowing modification of internal fields (`access_tier`) that are never exposed through the legitimate UI.

Root Cause

Two independent failures compound each other:

1. Missing object-level authorization check — the endpoint never verifies that `req.user.employee_id` matches the `employee_id` being modified.
2. Missing input whitelisting — the update handler blindly assigns all incoming keys onto the target object instead of only accepting an explicit set of permitted fields.

Recommended Fix

- Derive the target record exclusively from the verified JWT (`req.user.employee_id`); never accept a client-supplied identifier for "my own" resource updates.
- If updating another user's resource is a legitimate feature, enforce an explicit authorization check (e.g. role-based check) before allowing it, in addition to a proper audit log.
- Replace the blind merge with an explicit whitelist of updatable fields (e.g. only `display_name`). Reject or silently ignore any other field in the payload.

Required Security Controls

- Object-level authorization check on every request that operates on a specific resource ID.

- Server-side input validation with an explicit allow-list per endpoint (never a deny-list).
- Separation between "public-facing" fields and internal/privileged fields at the data-model level, not just at the API response layer.

Expected Secure Behavior

A request attempting to modify `employee_id` values other than the caller's own is rejected (403/404), regardless of which fields are included in the body. Fields outside the explicit whitelist (e.g. `access_tier`) are silently dropped even when the caller is modifying their own record.

Retest Procedure

1. Authenticate as a low-privilege user.
 2. Send `POST /api/profile/preferences` with `employee_id` set to a different valid user's ID and any payload.
 3. Confirm the request is rejected and no fields on the target record change.
 4. Send the same request targeting the caller's own `employee_id`, including a non-whitelisted field (e.g. `access_tier`).
 5. Confirm the non-whitelisted field is not persisted, while whitelisted fields (e.g. `display_name`) still update correctly.
-

2. GraphQL — Broken Object Level Authorization (Legacy Resolver)

Vulnerable Behavior

A legacy GraphQL endpoint (`/graphql/v1`) remains deployed alongside the current, hardened endpoint (`/graphql/v2`). The legacy schema exposes a resolver, `legacyCredentialResolver(username)`, which returns sensitive account data for whichever `username` is passed as an argument, without verifying that the requesting user is the owner of that account. The legacy endpoint also has introspection enabled, allowing full schema enumeration, while the current endpoint correctly disables it.

Root Cause

- The legacy API version was never decommissioned after the v2 migration, leaving an unreviewed, unmaintained attack surface live in production.

- The resolver was originally written for a trusted, internal-only migration tool and never had an object-level ownership check added when it was (unintentionally) left reachable externally.
- Introspection was left enabled on a version of the schema that was assumed to be "internal only," rather than being disabled uniformly across all environments.

Recommended Fix

- Fully decommission and remove legacy API versions once a replacement is in production. Do not leave old versions reachable "just in case."
- Add an explicit ownership check to any resolver that accepts an identifier as an argument: `context.user.username === args.username` (or an equivalent role check for legitimate cross-account access).
- Disable introspection uniformly across all environments and all API versions, not just the current one.

Required Security Controls

- API version lifecycle management/deprecation policy with enforced sunset dates.
- Field-level and resolver-level authorization checks in GraphQL, equivalent to endpoint-level checks in REST.
- Consistent security configuration (introspection, rate limiting, auth requirements) applied identically across all API versions.

Expected Secure Behavior

Any request to `LegacyCredentialResolver` (or its equivalent in a maintained API) for a `username` other than the caller's own is rejected. Introspection queries against any deployed GraphQL endpoint return a "disabled" error rather than schema data.

Retest Procedure

1. Authenticate as a low-privilege user.
2. Query `LegacyCredentialResolver` (or its replacement) for the caller's own username — confirm it returns only the caller's own data.
3. Query the same resolver for a different, valid username — confirm the request is rejected.

4. Send an introspection query (`{ __schema { types { name } } }`) against every live GraphQL endpoint — confirm introspection is disabled everywhere.
-

3. gRPC — JWT Authentication Bypass + Proto File Disclosure

Vulnerable Behavior

The `AdminService` gRPC interceptor accepts a JWT supplied in call metadata and decodes it without verifying its signature. Specifically, if the token header specifies `"alg": "none"`, the interceptor trusts the payload's claims (including a `role` claim) without any cryptographic verification, allowing a fully self-issued, unsigned token to be treated as authentic.

Root Cause

The authentication logic manually decodes the JWT and special-cases the `none` algorithm instead of using a library-enforced allow-list of algorithms and mandatory signature verification. This is a known, historically common class of JWT implementation flaw.

Recommended Fix

- Always verify JWT signatures using a fixed, explicit list of accepted algorithms (e.g. `['HS256']`) passed to the verification library — never allow `none` to reach the verification path under any circumstance.
- Reject any token whose header algorithm does not exactly match the expected value before attempting to read any claims.
- Use a well-maintained JWT library's built-in verification function end-to-end, rather than manually parsing/decoding tokens and only partially verifying them.

Required Security Controls

- Enforced allow-list of JWT signing algorithms at the library/framework configuration level.
- Signature verification as a mandatory, non-bypassable step before any claim is trusted, for every authenticated RPC method.
- Least-privilege defaults: sensitive methods (e.g. `GetPasswordFragment`) should fail closed on any authentication ambiguity.

Expected Secure Behavior

A gRPC call carrying a token with `"alg": "none"`, or any signature that does not verify against the server's secret, is rejected with `UNAUTHENTICATED` before any claim (including `role`) is read or used in an authorization decision.

Retest Procedure

1. Construct a JWT with `"alg": "none"` and an empty signature segment, containing an arbitrary `role: admin` claim.
 2. Call `GetSystemStatus` and `GetPasswordFragment` with this token in the request metadata.
 3. Confirm both calls are rejected with `UNAUTHENTICATED`.
 4. Repeat with a token signed using an incorrect/arbitrary secret (valid `HS256` header, invalid signature) and confirm it is also rejected.
 5. Confirm that only a token correctly signed with the server's actual secret and containing `role: admin` is accepted.
-

4. Unintended Attack Paths Discovered During Testing

Two unintended shortcuts were identified during internal testing and have been remediated. Both are documented here per the challenge integrity requirements.

4.1 Authentication Bypass via Null Password Comparison

Vulnerable Behavior

The login handler compared the supplied password to the stored password using strict inequality (`employee.password !== password`). The `admin` account's stored password was `null` (intentionally, as it has no valid password). A login request submitting a literal JSON `null` for the password field satisfied `null !== null → false`, causing the comparison to pass and issuing a valid token for the `admin` account with no exploitation required.

Impact

This completely bypassed the intended REST BOLA exploitation path — an attacker could authenticate directly as `admin` without ever discovering or exploiting the object-level authorization flaw, undermining the primary learning objective of the REST stage.

Fix Applied

The login handler now explicitly requires both the stored and supplied password to be non-empty strings before comparing them, rejecting `null`, `undefined`, and empty-string values outright regardless of the stored password value.

Retest Procedure

Attempt login with `{"username": "admin", "password": null}`, with the `password` field omitted entirely, and with `password: ""`. Confirm all three attempts return `401 Unauthorized`. Confirm legitimate login with a valid username/password pair still succeeds.

4.2 Static File Exposure Bypassing the Backup Directory Access Gate

Vulnerable Behavior

A directory-listing misconfiguration was intentionally introduced on a `/backup` route, gated behind completion of the REST and GraphQL stages. However, the leaked file's physical location remained inside the generally-served static assets directory. The generic static file server, mounted separately at the application root, served the same file at its original filesystem-relative path with no gating check applied, completely bypassing the intended access restriction.

Impact

A player could retrieve the leaked service definition file needed for the gRPC stage without ever completing the REST or GraphQL stages, breaking the intended chaining requirement that gRPC access depends on prior progress.

Fix Applied

The leaked file was relocated outside of the publicly-served static assets directory entirely, into a location with no route exposing it except the explicitly gated `/backup` route. The generic static file server no longer has any path to the file.

Retest Procedure

Before completing REST and GraphQL, request the file via its original (pre-fix) path and confirm it is no longer served (falls through to the generic 404/fallback page). Request it via the gated `/backup` route and confirm `403 Forbidden`. After completing REST and GraphQL, confirm the `/backup` route now serves the file and directory listing correctly.